CS 4240 Project 2 - Optimizer Design

1 High-Level Architecture

Our optimizer takes in as input an intermediate representation for the Tiger Language, denoted with example.ir as input. Within our program, this Tiger Intermediate Representation passes through the pipeline of our optimizer as shown in the following Figure 1:

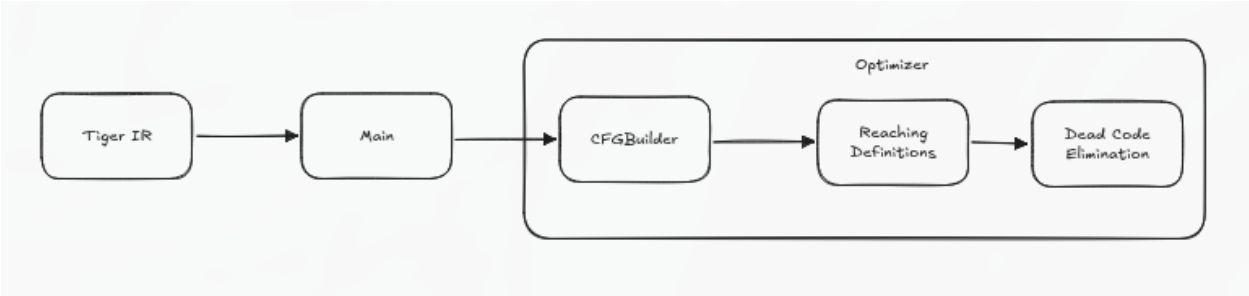


Figure 1. High-level pipeline of the optimizer

Main Class

When Main first receives the Tiger IR file, it parses the IR file into an IRProgram ready to be optimized. The IRProgram is a class that contains a List of IRFunction(s) that need to be optimized. With this IRProgram, we pass it into the IROptimizer class, which takes in the IRProgram and begins applying optimizations (as seen below).

```
public class Main {
  public static void main(String[] args) throws FileNotFoundException, IRException {
    if (args.length < 1) {
      System.err.println("Usage: Main <path-to-ir-file>");
      System.exit(1);
    }

    IRProgram program = new IRReader().parseIRFile(args[0]);
    new IROptimizer().optimize(program);
    new IRPrinter(System.out).printProgram(program);
  }
}
```

Figure 2. Entry into theIROptimizer class from Main

IROptimizer

The IROptimizer class is the main orchestrator of the different layers used for the Tiger IR optimization. Before any optimizations can be applied, a Control Flow Graph (CFG) must be made for each function in the IRProgram. This is done by the CFGBuilder class. The output of

the CFGBuilder class is a CFG. Once the CFG is created, it is then used to compute the ReachingDefinitions, which is needed to identify which variable assignments are able to reach a specific program point without being redefined. This ReachingDefinition object is then passed to a newly instantiated DeadCodeElimination. An optimized Mark and Sweep algorithm is run iteratively on the DCE object until no more changes can be made.

```
public IRProgram optimize(IRProgram program) {
  CFGBuilder builder = new CFGBuilder();
  for (IRFunction function : program.functions) {
    CFG cfg = builder.buildCFG(function);
    ReachingDefinitions rd = new ReachingDefinitions(cfg);
    DeadCodeElimination dce = new DeadCodeElimination(rd);

    boolean changed = true;
    while (changed) {
      changed = dce.run(cfg);
    }
    function.instructions = flatten(cfg);
  }
  return program;
}
```

Figure 3. IROptimizer class driver code

CFGBuilder

The CFGBuilder simply takes an IRFunction object, and then constructs a CFG from it using the buildCFG method. During its execution, BasicBlock(s) are created and populated starting with the very first instruction, and then continuing to populate a BasicBlock until a branch instruction, or a label is encountered. Then mappings are made between labels and their respective BasicBlock(s), to be used for determining who the successors (BasicBlocks that a BasicBlock may jump to) and predecessors (BasicBlocks who may have jumped to or who preceded you in execution). This creates a form of adjacency list of BasicBlocks that comprise our CFG.
Files: CFG.java, CFGBuilder.java, BasicBlock.java
See Figure 7 at the end for Visuals :)

ReachingDefinitions

The ReachingDefinitions class is used to track which variable assignments can reach a specific program point without being redefined somewhere else. It first computes the entire set of variable writes within the CFG (any instructions that write to a variable). Then, for each basic block, GEN/KILL and IN/OUT sets are computed iteratively until no changes can occur in the IN/OUT sets.
File: ReachingDefinitions.java

DeadCodeElimination

The DeadCodeElimination class has a run method which starts the actual process of removing dead code from the available instructions in the IRFunction. It first makes a call to the

ReachingDefinitions().run() function to start computing the IN/OUT sets necessary for the optimized mark and sweep. Within DeadCodeElimination's run() we determine for each instruction, what other instructions provide the values that it is reading. We also determine which instructions are actually needed, starting with everything that can't be removed (branches, stores, calls, etc.). For each of these instructions, dependencies are looked up within a map and are marked as not to be removed as well. The ReachingDefinitions object is initially used to populate the set of live instructions coming from other BasicBlocks in the CFG, allowing our program to accurately create the worklist based on flows across the entire CFG. These are used to compute a toRemove List used to remove useless instructions from the IRFunction.
File: DeadCodeElimination.java

```
public boolean run(CFG cfg) {
  rd.run(cfg);
  Map<IRInstruction, Set<IRInstruction>> depMap = computeDependencies(cfg);
  // The Mark Pass
  Set<IRInstruction> markedInsts = markLiveInstructions(cfg, depMap);
  boolean changed = false;
  for (BasicBlock block : cfg.getBlocks()) {
    List<IRInstruction> toRemove = new ArrayList<>();
    for (IRInstruction inst : block.instructions) {
      if (Helpers.defines(inst) && !markedInsts.contains(inst)) {
        if (inst.opCode != IRInstruction.OpCode.CALLR) {
          toRemove.add(inst);
          changed = true;
        }
      }
    }
    block.instructions.removeAll(toRemove);
  }
  return changed;
}
```

Figure 4. DeadCodeElimination run function handling dead code removal

Rational

The design follows what was taught in class. We especially relied heavily on the pseudocode provided in Lecture 8. We started by implementing simple mark and sweep and dead code elimination. This was the baseline optimization (optimizer A). This approach works well if there are no branches or gotos in the code, which is usually not the case. However, with control flow, a control flow graph and reaching definitions is needed to better optimize the IR. There are other methods, like static single assignment, that can be used to optimize the IR, instead of reaching definitions; however, they are more complex and not necessary.

2 Low-Level Design Decisions

We used Java for this project. Not only do all team members have experience with this language, but the definitions for the Intermediate Representations were also provided in Java,

making it easier to start implementing the optimizations.The key data structures to note are the classes that represent each of the components needed for optimization.

- BasicBlock: This class is the basis of our CFG abstraction holding the instructions contained within the block, as well as predecessors and successors (edges between the blocks)
- CFG/CFGBuilder: The CFGBuilder class contains the methods that build the CFG abstraction, while the CFG class just holds the List of BasicBlocks that comprise the CFG
- ReachingDefinitions: This class holds all of the computed sets (GEN/KILL/IN/OUT/) necessary for our enhanced mark and sweep algorithm
- DeadCodeElimination: This is the class that actually performs the changes on the IRFunction instructions utilizing our IN set from ReachingDefinitions as well as the mark and sweep worklist algorithm
- Helpers: This file just contains some helper methods that were useful throughout the project to reduce repeating ourselves when writing code (DRY!)
- IROptimizer: This class is what coordinates all of our different classes together to optimize a passed in IRProgram

3 Software Engineering Challenges

One of the earliest challenges we encountered was with refactoring our program to add more functionality. We originally tightly coupled both CFG generation and mark/sweep within a singular CFG.java class. This however made it difficult for other team members to refactor and add new functionality to (like reaching definitions) given the large difference in the data structures needed for the CFG and the IN/OUT sets, etc. We quickly started to face the possibility of class bloat. To combat this, we applied SOLID principles (mostly just the S). The S in SOLID stands for the Single-Responsibility Principle. Officially this means," A class should have one and only one reason to change, meaning that a class should have only one job." This makes it easier to both extend and debug an application when it's broken up into composable parts. By creating a class for each part of our optimizer pipeline, that does a specific function (that can be easily identified by its class name what its likely used for) we were able to add more functionality easier by simply creating a new class, and then adding that new class to our pipeline defined in the IROptimizer. If for instance, we wanted to add Constant Folding and/or Constant Propagation, it would be as easy as creating new classes to implement both (since we have the BasicBlock abstraction and CFG in their own respective classes) and adding them to the IROptimizer's run function as well.

Known Bugs/Deficiencies

As of the time of writing this, there are no known implementation bugs, and our implementation meets the requirements for dynamic instruction counts across public tests and internal team testing.

Build & Usage

To build and use our program, you simply run:

```
…/Project-2  main $  23:49
❯ ./build.sh
Build successful.

…/Project-2  main $  23:49
❯ ./run.sh <path-to-ir>
```

Figure 5. Program build and run shell example

Test Results Summary

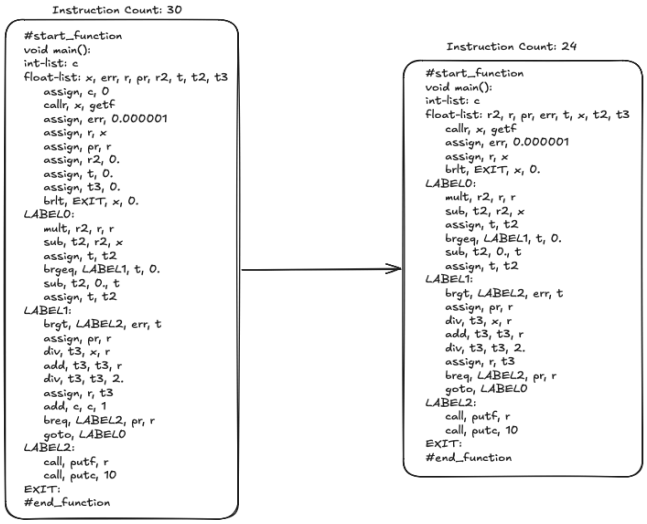


Figure 6. Optimized mark and sweep output for the sqrt.ir passed in

Initially, there are 30 instructions in sqrt intermediate representation. After the optimization algorithms run, the static instruction count goes down to 24. This is a 20% reduction in the number of instructions in the intermediate representation. The other publicly available test quicksort.ir gave us a more modest reduction, ~8% in the number of instructions (Table 1).. There is also a decrease in the number of dynamic instructions. Originally, for sqrt ir, with an input of 25, the dynamic instruction count is 94. After DCE with RD runs, it is 83 (Table 2). For quicksort, for input 0.in, the dynamic instruction count is 482; after, it is 434, which is a 9.96% reduction (Table 3). As you can see, there is an increase in performance using our optimizer.

Table 1. Static Instruction Count

Program	# of Static Instructions Before	# of Static instructions After	Percent Reduction
sqrt	30	24	20%
quicksort	52	48	8%

Table 2. Sqrt Dynamic Instruction Count

Test	# of Dynamic Instructions Before	# of Dynamic Instructions After	Percent Reduction
0	94	83	11.7%
1	70	61	12.86%
2	16	11	31.25%

Table 3. Quicksort Dynamic Instruction Count

Test	# of Dynamic Instructions Before	# of Dynamic Instructions After	Percent Reduction
0	482	434	9.96%
1	1124	1026	8.72%
2	1806	1658	8.19%

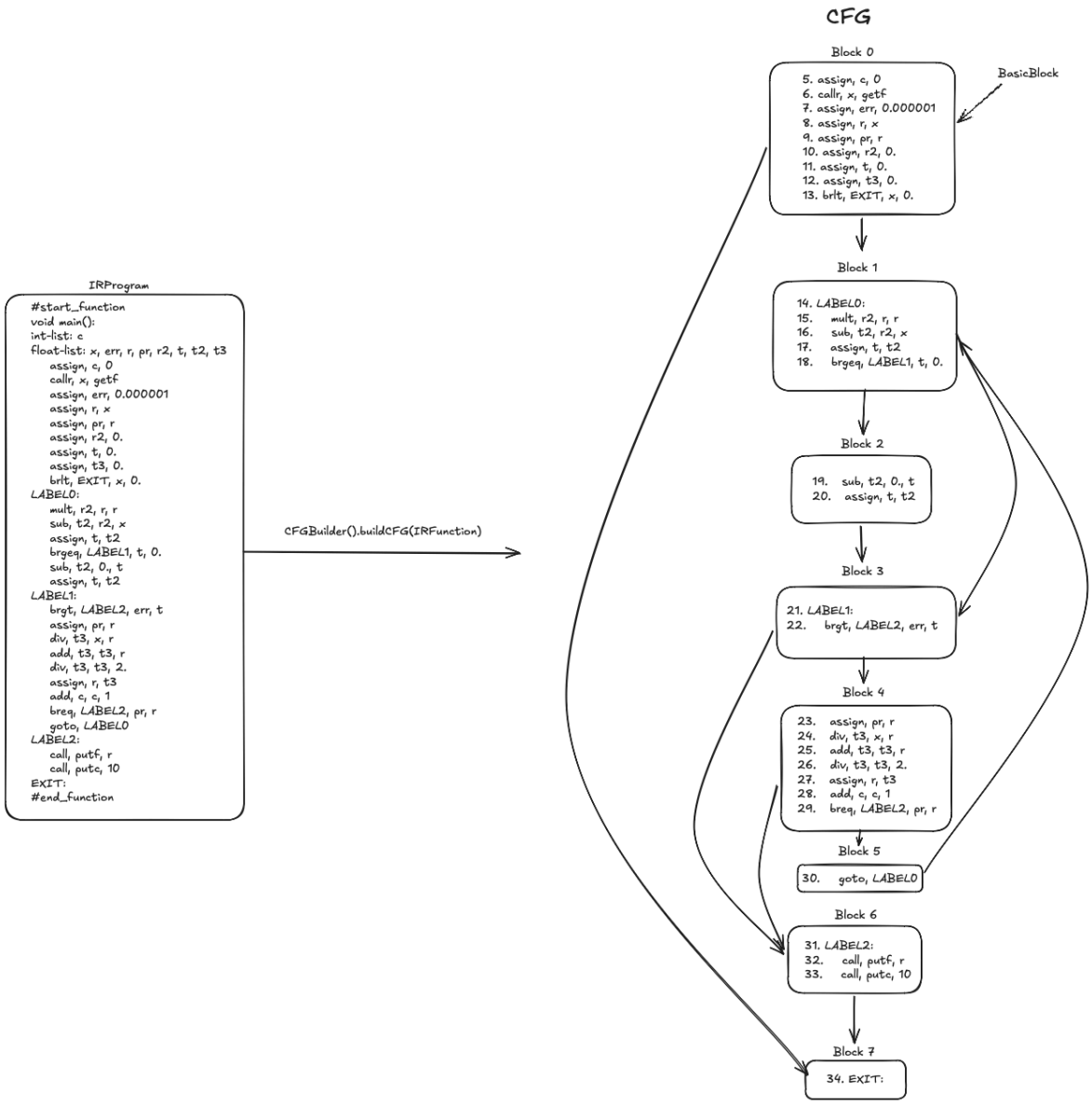


Figure 7. Conversion of Intermediate Representation to Control-Flow Graph comprising Basic Blocks